

# Tighter Approximation Schemes for Chain-Graph Logical Credal Networks

IBM Research

## Abstract

A chain-graph Logical Credal Network (LCN) admits a factorization that captures its Local Markov Condition (LMC) independence assumptions. The current `cn/` pipeline exploits this by compiling the LCN into a *credal network*: a directed graph whose local credal sets  $K(\text{child} \mid \text{parents})$  are interval-valued and computed per family, then performing posterior inference (Credal Variable Elimination) over the *strong extension* of that credal network. The strong extension is generally a *proper superset* of the LCN’s distribution set, so the computed posterior marginal bounds are looser than the exact LCN bounds. This note dissects the two distinct sources of this looseness on a running example (the `alarm` network), proposes five approximation schemes that produce tighter outer approximations—hence tighter posterior bounds—illustrates each on the example, explains how the existing inference algorithms (Credal VE, Interval BP, ApproxLP, Credal CTE, Credal IJGP) all inherit the tightening, and gives a validation strategy based on certified local solves and the existing LMC factorization verifier.

## 1 Background and notation

Let an LCN over propositional atoms  $\mathbf{V} = \{V_1, \dots, V_n\}$  define a set of joint distributions

$$\mathcal{D}_{\text{LCN}} = \{ P \in \Delta(2^{\mathbf{V}}) : P \text{ satisfies every sentence } s \},$$

where each sentence  $s$  is a Type-1 constraint  $\ell \leq P(\varphi) \leq u$  or a Type-2 constraint  $\ell \leq P(\varphi \mid \psi) \leq u$ . When the LCN’s structure graph is a chain graph, the joint factorizes over the families (child + parents) of the simplified structure graph,

$$P(\mathbf{V}) = \prod_v P(\text{ch}_v \mid \text{pa}_v), \tag{1}$$

and this factorization reproduces exactly the LMC independencies (cf. `docs/ISIPTA2025_LCNs.pdf` Sec. 3, `docs/LCN_IJAR_Revised.pdf`).

The `cn/` pipeline realizes (1) in three stages, each of which is implemented in a distinct module:

1. **Symbolic factorization** (`cn/factorization.py`): enumerate the families and record, per family, the child, parents, flattened atom scope and the LCN sentences *attached* to that family.
2. **Local credal sets** (`cn/local_credal_sets.py`): for each family and each interpretation of its scope, solve a (fractional) optimization problem to obtain the lower/upper bound  $[p, \bar{p}]$  of the conditional  $P(\text{ch}_v = c \mid \text{pa}_v = \pi)$ . The method axis carries exactly two settings (Section 1.1): `method="linear"` (the baseline) treats each family *in isolation*, and `method="linear-tight"` solves the same family against every in-scope LCN sentence plus the scope-restricted LMC equalities (scheme D1 of Section 4).

3. **Inference over the strong extension** (`cn/vertices.py`, `cn/cve.py`): enumerate the extreme points of each local credal set and run Credal Variable Elimination, which optimizes over the strong extension—each local credal set, for each parent configuration, contributes a freely and independently chosen extreme point.

### 1.1 Build API: two methods, two orthogonal knobs

The schemes below are exposed through the existing build entry points (`CredalNetwork.from_lcn`, `CredalNetworkVertices.from_lcn`) without multiplying flags. The previous `method="nlp"` option (which added only *pairwise* parent marginal independence) is **removed**: it is a strict half-measure for source A, dominated by D1 below, and its bilinear program added solver fragility for little tightening. The surface is therefore:

- `method`  $\in \{\text{"linear"}, \text{"linear-tight"}\}$ . "linear" is today's per-family-in-isolation baseline (unchanged). "linear-tight" is scheme D1: each family is solved against every LCN sentence whose atom scope lies inside the family scope, plus the scope-restricted LMC bilinear equalities.
- `solver`  $\in \{\text{"ipopt"}, \text{"scip"}\}$  (already present). On "linear-tight", `solver="scip"` is scheme D3: it solves the enriched local program to certified global optimality; "ipopt" is the fast, hardened (multi-restart + SLSQP-fallback) variant of the same program.
- `merge_budget`  $k$  (new): scheme D2. A pre-factorization pass merges adjacent families whose combined scope is  $\leq k$  before `ChainGraphFactorization.build`;  $k=1$  is no merge (today),  $k = \text{full scope}$  is exact. Orthogonal to `method` and `solver`.

Thus D1 is the *core* of "linear-tight", while D2 ( $k$ ) and D3 (`solver`) are independent cost-vs-tightness dials rather than separate method names. (This `cn/` factorization `method` is unrelated to the identically named parameters of `ibp.py` and `map/exact_map.py`.)

## 2 Running example: the alarm network

We use `examples/alarm.lcn` throughout. It has five atoms  $\{A, B, C, D, E\}$  and five sentences:

```
s1: 0.1  <= P(B)                <= 0.2
s2: 0.05 <= P(E)                <= 0.1
s3: 0.6  <= P(A | (B or E))     <= 0.9
s4: 0.5  <= P(not(C xor D) | A) <= 0.7   ; True
s5: 0.05 <= P(A)                <= 0.1
```

Its structure is a chain graph. The simplified structure graph has the compound node  $C-D$  (because  $s4$  ties  $C$  and  $D$  together through  $C \oplus D$ ), so the families and the LMC independencies the pipeline derives are:

| Family           | Attached sentences | LMC independencies         |
|------------------|--------------------|----------------------------|
| $P(A \mid B, E)$ | $s1, s2, s3, s5$   | $(B \perp E)$              |
| $P(B)$           | $s1$               | $(C \perp B, E \mid D, A)$ |
| $P(E)$           | $s2$               | $(D \perp B, E \mid C, A)$ |
| $P(C-D \mid A)$  | $s4, s5$           |                            |

The factorization is  $P(A \mid B, E) P(C-D \mid A) P(B) P(E)$ .

**Local credal sets computed today (method="linear").** Solving each family in isolation gives the conditionals below (lower/upper, rounded). Note the many vacuous  $[0, 1]$  entries—these are exactly where gap A bites.

| $P(A \mid B, E)$                    |                | $P(C - D \mid A)$     |            |
|-------------------------------------|----------------|-----------------------|------------|
| $A=0, B=0, E=0$                     | $[0.956, 1.0]$ | $C=0, D=0, A=1$       | $[0, 0.7]$ |
| $A=1, B=0, E=0$                     | $[0, 0.044]$   | $C=0, D=1, A=1$       | $[0, 0.5]$ |
| $A=0, B=0, E=1$                     | $[0, 1.0]$     | $C=1, D=0, A=1$       | $[0, 0.5]$ |
| $A=1, B=0, E=1$                     | $[0, 1.0]$     | $C=1, D=1, A=1$       | $[0, 0.7]$ |
| $\dots$ (rows with $B=1$ or $E=1$ ) | $[0, 1.0]$     | $\dots$ ( $A=0$ rows) | $[0, 1.0]$ |

Two phenomena stand out and recur in the discussion below:

- **s3 only constrains the  $(B \vee E)$  corner.** s3 bounds  $P(A \mid B \vee E)$ , i.e. the conditional given the *disjunction*. It says nothing directly about  $P(A \mid B=0, E=1)$  or  $P(A \mid B=1, E=0)$  individually, so those parent configurations come back vacuous  $[0, 1]$ —even though, jointly with s5 ( $P(A) \in [0.05, 0.1]$ ) and the marginals s1, s2, the LCN pins them far tighter.
- **s4 constrains a sum, not a single state.** s4 bounds  $P(\neg(C \oplus D) \mid A) = P(C=D \mid A)$ , i.e. the sum of the two diagonal states  $P(C=0, D=0 \mid A) + P(C=1, D=1 \mid A)$ . The isolated family LP can therefore push any single state to its extreme (hence  $[0, 0.7] / [0, 0.5]$ ), and for  $A=0$  there is no constraint at all, giving  $[0, 1]$ .

This is the concrete object the schemes of Section 4 tighten.

### 3 Two sources of looseness

Write  $\tilde{\mathcal{D}}$  for the joint credal set actually optimized by CVE. We have  $\mathcal{D}_{\text{LCN}} \subseteq \tilde{\mathcal{D}}$ , and the inclusion is typically strict for *two independent reasons*.

#### 3.1 (A) Local-set widening

For family  $v$  the pipeline computes

$$K_v^{\text{local}} = \left\{ P(\text{ch}_v \mid \text{pa}_v) : \text{the sentences attached to family } v \text{ hold} \right\},$$

because `_linear_rows_and_obj` loads only `family["sentences"]`. The true projection of the LCN onto that family is

$$K_v^{\text{true}} = \left\{ P(\text{ch}_v \mid \text{pa}_v) : \exists P \in \mathcal{D}_{\text{LCN}} \text{ inducing this conditional} \right\},$$

and  $K_v^{\text{true}} \subseteq K_v^{\text{local}}$ , often strictly: sentences attached to *other* families and the LMC equalities further constrain the conditional but are discarded. `method="nlp"` repairs only one slice of this gap (pairwise parent marginal independence) and is still loose for  $\geq 3$  parents and for any constraint that spans families.

*On the example.* The  $[0, 1]$  entries of  $P(A \mid B=0, E=1)$  and of  $P(C - D \mid A=0)$  are pure gap-A artifacts: s5 ( $P(A) \leq 0.1$ ) and the parent marginals s1, s2 are not seen when family  $A$  is solved against only its attached rows, and family  $C - D$  never sees that  $A$  is rare. The true projections are far narrower.

### 3.2 (B) Strong-extension widening

Even with  $K_v = K_v^{\text{true}}$  exactly, the strong extension

$$\mathcal{D}_{\text{SE}} = \left\{ \prod_v P_v : P_v \in K_v \text{ chosen independently per parent-config} \right\}$$

allows every (node, parent-configuration) slot to pick its extreme point freely. The LCN’s non-local sentences *couple* choices across families that  $\mathcal{D}_{\text{SE}}$  decouples, so  $\mathcal{D}_{\text{LCN}} \subsetneq \mathcal{D}_{\text{SE}}$  in general. This is the credal set whose product structure `verify_lmc_factorization.py` already reconstructs (its `_JointReconstructor` takes one extreme point per (node, parent-config) slot and forms the product joint).

*On the example.* Gap B shows up in the  $C-D$  family. The strong extension lets the four states of  $P(C-D \mid A=1)$  be chosen so that  $P(C=0, D=0 \mid A=1)$  takes its max 0.7 and  $P(C=1, D=1 \mid A=1)$  takes its max 0.7 in the same joint—a vertex combination whose diagonal sum is 1.4, violating  $s4$ ’s upper bound of 0.7. The LCN forbids that distribution; the strong extension admits it. No tightening of the *intervals* removes it, because the constraint couples two states of the same factor.

Either gap alone suffices to make posteriors loose; a tight scheme must close the one that dominates on the instance at hand.

## 4 Proposed schemes

**D1 — Constraint back-propagation into local sets (attacks A; the new "linear-tight" method).** When solving family  $v$ , replace the constraint source from “sentences attached to  $v$ ” with “every LCN sentence  $s$  whose atom scope satisfies  $\text{atoms}(s) \subseteq \text{scope}(v)$ ,” and additionally append the LMC bilinear equalities (`lmc_constraint_groups.vec` in `utils/common.py`) restricted to  $\text{scope}(v)$ . This is exactly `method="linear-tight"`: when the enriched constraint set is purely linear the family stays an LP / Charnes–Cooper fractional LP, and when it acquires LMC bilinear equalities it is solved on the existing hardened ipopt (or, under `solver="scip"`, certified-global) path that the removed `"nlp"` option used to drive. Closes the part of gap A expressible within a single family scope; cannot capture constraints spanning atoms in different families. Near-zero structural change—only `_linear_rows_and_obj`’s sentence selection and a scope-restricted LMC row builder.

*On the example.* Family  $A$ ’s scope is  $\{A, B, E\}$ , so  $s1, s2, s5$  all satisfy  $\text{atoms}(s) \subseteq \text{scope}$  and would now be loaded together with  $s3$ . The disjunctive  $s3$  plus  $s5$  ( $P(A) \leq 0.1$ ) plus the parent marginals collapse the vacuous  $[0, 1]$  rows of  $P(A \mid B, E)$  to genuine sub-intervals. Family  $C-D$  now sees  $s5$ , tightening the  $A=0$  rows. The diagonal-sum coupling of  $s4$  is *within* the  $C-D$  family scope, so this single change also removes the gap-B vertex combination of Section 3 from that local set.

**D2 — Scope merging with a tightness dial (attacks A, partially B; the `merge_budget` knob).** Introduce a budget  $k$  (`merge_budget`). Merge adjacent families whose combined scope size is  $\leq k$  into a single *super-family* and compute one joint local credal set over the merged scope (a larger fractional LP / NLP). At  $k = \text{full scope}$  this is exact inference; at  $k = \text{single-family}$  it is today’s behavior. Provides an anytime tightness-vs-cost knob, with per-super-family cost  $2^k$ . Implemented as a merging pass over `lcn.families` before `ChainGraphFactorization.build`, reusing the existing per-family solver on the larger scope; `estimate_difficulty` can choose  $k$  adaptively.

*On the example.* Merging families  $A$  and  $C-D$  (they share the edge  $A \rightarrow C-D$ ) into a super-family over  $\{A, B, E, C, D\}$  makes  $s3, s4, s5$  act jointly: the bound on  $P(C=D \mid A)$  is now solved in a

model that also knows  $P(A) \leq 0.1$  and the  $A \mid B, E$  structure, so the  $C-D$  conditionals tighten beyond what D1 achieves in isolation. Here  $k=5$  is the full scope, i.e. exact; a budget  $k=3$  would merge only where cheap.

**D3 — Joint-LP certification of each local bound (attacks A; exact, certified; the solver="scip" path).** For each (family, interpretation, sense), solve the bound for the family conditional  $P(\text{ch}_v \mid \text{pa}_v)$  against the *full* LCN feasible region: all LCN sentences and all LMC equalities over the  $2^n$  joint, with a fractional objective local to the family. This yields  $K_v^{\text{true}}$  exactly. It is reached by combining `method="linear-tight"` with `solver="scip"` (the existing global path) so the bounds carry a global optimality certificate. Cost is  $2^n$  variables per solve; viable for small  $n$  or, more usefully, as the *certifier* of the cheaper schemes (it is essentially `ExactInference` run per conditional).

**D4 — Region-restricted (extensive) credal network (attacks B; structural).** Replace the free per-parent-config vertex choice with a *coupled* one. Variants: (4a) detect sentences that couple multiple families and add constraints in `cve.py`'s bucket elimination forbidding vertex combinations that violate them; (4b) introduce a shared latent "regime" parent to each coupled family so the local credal sets become conditional on a common selector, restoring the cross-family correlation the LCN imposes. Touches the CVE elimination, not the local solves.

*On the example.* D4 directly kills the *s4* violation of Section 3: the four extreme points of  $P(C-D \mid A=1)$  may no longer be combined into a joint whose diagonal sum exceeds 0.7. Under variant 4a, CVE's bucket for  $C-D$  carries the side constraint  $P(C=0, D=0 \mid A=1) + P(C=1, D=1 \mid A=1) \leq 0.7$  across the vertex choice, restoring the coupling that the free product had dropped.

*Implementation note (what D4 can and cannot couple).* The *s4* example above is, on inspection, an *in-family* constraint ( $\text{atoms}(s4) = \{C, D, A\} \subseteq$  the  $C-D$  family scope), so D1 already removes it; it is shown only to illustrate the coupling mechanism. The constraints D4 must actually handle are the *cross-family* ones — those whose atom set is a subset of no single family scope. A subtle but important fact constrains the menu: the strong extension of a chain-graph credal network is a *product*  $\prod_v P(\text{ch}_v \mid \text{pa}_v)$ , and that product structure *satisfies every LMC independence by construction* — which is exactly why `verify_lmc_factorization.py` reports the factorization SUPPORTED on every chain LCN we have tried. Consequently *no* product joint can violate a cross-family *LMC assertion*: enforcing them in D4 is sound but inert. The gap-B looseness D4 removes is therefore carried by cross-family *sentences* (Type-1/Type-2 bounds whose scope spans families), which the free product *can* violate because each local factor is chosen to satisfy only its own attached sentences. As a concrete witness, a three-family instance  $P(A), P(B \mid A), P(C \mid A)$  with the cross-family sentence  $P(B \wedge C) \leq 0.05$  admits a free-product joint reaching  $P(B \wedge C) = 0.92$ ; D4 forbids it, collapsing e.g.  $P(B \mid C=1)$  from  $[0.04, 0.96]$  to a near-point. The implementation (`cn/coupling.py`, the `coupling="cross-family"` flag on Credal VE / CTE / ApproxLP) classifies a constraint as cross-family against the *built* factors, so a D2 `merge_budget` that fuses the coupled families turns the same constraint in-family and removes it from D4's list — D2 and D4 compose without double-counting.

**D5 — Constraint-aware CVE (attacks A and B; the tight target).** Keep the chain-graph structure for the elimination order and bucket scopes, but at each bucket combination optimize the bucket potential subject to the original LCN sentences *projected onto the bucket scope*, rather than over the free strong-extension vertices. This is a sequence of small NLPs threaded through variable elimination—a credal VE that respects the global constraints at every step and never materializes

| Scheme                                 | Source closed | Tightness                  | Cost               | Effort  |
|----------------------------------------|---------------|----------------------------|--------------------|---------|
| D1 back-propagate in-scope constraints | A (partial)   | +                          | $\approx 0$        | low     |
| D2 scope merging (dial $k$ )           | A (+ B)       | dial                       | $2^k$              | low-med |
| D3 joint-LP certified locals           | A (exact)     | ++                         | $2^n/\text{solve}$ | med     |
| D4 region-restricted CVE               | B             | +                          | med                | med     |
| D5 constraint-aware CVE                | A + B         | +++ ( $\rightarrow$ exact) | treewidth          | high    |

Table 1: Summary of the proposed schemes against the two looseness sources of Section 3.

the loose strong extension. In the limit it equals exact inference but its cost is bounded by the largest bucket (treewidth), not  $2^n$ .

*On the example.* Eliminating  $B, E$  first produces a bucket over  $\{A\}$  whose potential is optimized subject to  $s1, s2, s3, s5$  jointly (closing gap A for A); eliminating  $C, D$  uses a bucket over  $\{A\}$  optimized subject to  $s4$  as a coupled sum (closing gap B for  $C - D$ ). The query marginal  $P(A)$  is then read off the final  $\{A\}$  bucket. No step ever forms the loose product, and no bucket exceeds three atoms.

*Implementation note (junction-tree exact NLP).* D5 is realized (`cn/junction_nlp.py`, the `coupling="d5"` flag on Credal VE) not as a sequence of message NLPs but as a single equivalent *junction-tree* NLP, which is cleaner to make provably exact. Build a clique tree from the chain-graph elimination order (reusing the bucket-tree construction), introduce one variable vector per cluster ranging over that cluster’s joint distribution, tie adjacent clusters with Lauritzen separator-consistency equalities (the marginal of a cluster onto a separator equals the neighbour’s), and impose each LCN sentence and LMC equality on a cluster containing its atoms. Because the clique tree has the running-intersection property, a separator-consistent constraint-satisfying family of cluster marginals projects to exactly the marginals of some global joint in the LCN’s set, and conversely; hence min / max of the query marginal over this NLP equals the exact bound, at cost  $\sum_c 2^{|\text{atoms}(c)|}$  rather than  $2^n$ . Cross-family constraints (whose atoms span cliques) are handled by augmenting the elimination order with their node sets so they acquire a single host cluster — the JT analogue of D2’s family merging and D4’s order augmentation; the cluster a constraint forces sets the local treewidth, so on instances whose cross-family constraints are themselves near-global (e.g. `alarm`, whose LMC assertions  $C \perp B, E \mid D, A$  span all five atoms) D5 degenerates to near-full-joint cost, while on instances with local cross-family coupling it stays cheap.

Two findings from the implementation. (1) The cluster NLP is nonconvex (Type-2 ratios, the bilinear LMC equalities, and the conditional-query ratio objective), so a local solver (ipopt) only reaches a local optimum that can fall *short* of the true bound; D5 must use the certified global backend (SCIP) to actually be exact, and defaults to it. (2) D5 is exact for *conditional* queries  $P(q \mid e)$ , where the vertex-enumeration methods (plain Credal VE, and D4 on top of it) are *not even sound*: the conditional bound is a ratio whose extremum need not lie at a vertex of the unconditioned credal set, so min/max-over-vertices can land strictly inside the true interval. On `examples/d4.biting.lcn`,  $P(B \mid C=1)$  is exactly  $[0.006, 0.926]$  (D5, matching the global solver), whereas plain Credal VE reports  $[0.038, 0.963]$  and D4 reports the spurious near-point  $[0.038, 0.038]$ . D5 is therefore the only member of the family that is sound for conditional queries, not merely tighter.

## 5 Leverage by the other inference algorithms

The `cn/` package ships five inference algorithms beyond CVE. Crucially, *all of them consume the same object*: a built `CredalNetworkVertices`, i.e. the directed structure plus the enumerated extreme points of the local credal sets (`cnv.extreme_points`). They differ only in how they propagate those vertices:

| Algorithm                             | Propagation                   | Bound character vs. the strong extension                       |
|---------------------------------------|-------------------------------|----------------------------------------------------------------|
| Credal VE ( <code>cve.py</code> )     | bucket elimination            | exact (the strong-extension interval)                          |
| Credal CTE ( <code>ccte.py</code> )   | two-pass bucket tree          | exact ( $\epsilon=0$ ); outer if $\epsilon>0$                  |
| ApproxLP ( <code>approxlp.py</code> ) | per-variable LP over vertices | <i>inner</i> (subset of the true interval)                     |
| Interval BP ( <code>ibp.py</code> )   | loopy interval messages       | <i>outer</i> ( <b>interval</b> ); inner ( <b>variational</b> ) |
| Credal IJGP ( <code>ijgp.py</code> )  | join-graph, <i>i</i> -bound   | outer-style; tunable by <i>i</i> -bound                        |

The essential observation is that every one of these algorithms brackets the *strong-extension* interval  $[q_{SE}, \bar{q}_{SE}]$ , not the LCN interval  $[q_{LCN}, \bar{q}_{LCN}]$ . Since

$$[q_{LCN}, \bar{q}_{LCN}] \subseteq [q_{SE}, \bar{q}_{SE}],$$

shrinking the strong extension toward  $\mathcal{D}_{LCN}$  pulls every algorithm’s output inward. This splits the schemes into two leverage classes.

**Class 1 — input-level tightening (D1, D2, D3): free for all five algorithms.** D1/D2/D3 change only what `CredalNetworkVertices.from_lcn` produces—tighter local intervals, hence tighter enumerated vertices. Because the algorithms read those vertices without assuming how they were obtained, *no algorithm code changes*. The effects compose with each algorithm’s own character:

- **CVE / CCTE** (exact over the strong extension): their interval is exactly  $[q_{SE}, \bar{q}_{SE}]$ , so a tighter strong extension is reported verbatim—the cleanest beneficiary.
- **ApproxLP** (inner): its interval sits *inside* the strong-extension interval. Tighter vertices raise its lower bound and lower its upper bound too, but the inner gap remains; pairing ApproxLP (inner) with CVE (outer) on the *same* tightened vertices brackets the LCN truth from both sides.
- **Interval BP** (outer): tighter local messages mean tighter fixed-point intervals; the loopy outer slack is unchanged but starts from a narrower base.
- **IJGP**: tighter cluster potentials propagate through the join graph; the *i*-bound still trades cost for tightness independently.

On the example, replacing the vacuous  $[0,1]$  rows (D1) with genuine sub-intervals narrows the vertex sets fed to *all five*, so e.g. a query  $P(A)$  tightens for CVE, ApproxLP, IBP, CTE and IJGP simultaneously, with one build-stage change.

**Class 2 — propagation-level coupling (D4, D5): per-algorithm work.** D4/D5 do not merely narrow intervals; they remove vertex *combinations* (D4) or avoid forming the product altogether (D5). This is a change to the propagation, so each algorithm needs its own adaptation:

- **CVE / CCTE**: add the cross-vertex side constraints (D4) to the bucket / cluster combination step. This is the most natural fit—both already optimize over vertex tuples per bucket, so a coupling constraint slots in directly. D5 is essentially a constraint-aware rewrite of CVE’s bucket optimization.
- **ApproxLP**: its per-variable LP already optimizes over a vertex polytope; D4’s coupling constraints can be added as extra LP rows, keeping the inner-bound guarantee while shrinking the polytope.
- **Interval BP / IJGP**: message passing does not carry global cross-factor constraints naturally. The practical route is to let D4’s coupling be absorbed at the *build* stage where possible (so it reduces to Class 1 for these two), and accept that the residual coupling D4 cannot localize is simply not captured by loopy / join-graph propagation—these remain the loosest of the family, by design.

**Practical recommendation.** Because Class 1 is free for all algorithms, apply D1 (and D2 where affordable) at the build stage *first*; every downstream algorithm benefits without modification. Reserve the per-algorithm D4/D5 work for CVE/CCTE (and ApproxLP), which can absorb the coupling exactly, and treat IBP/IJGP as fast outer approximations that ride on the Class-1 tightening.

## 6 Validation strategy

Tightening must be checked against ground truth on two fronts, matching the two gaps.

**Local-set tightness (gap A) via D3.** D3 is itself the certifier: for any candidate local credal set produced by D1 or D2, compare its interval  $[p, \bar{p}]$  for each (family, interpretation) against the D3 certified-global interval  $[p^*, \bar{p}^*]$  obtained from the SCIP joint-LP. A correct outer approximation requires  $[p^*, \bar{p}^*] \subseteq [p, \bar{p}]$ , and the width difference quantifies the residual local looseness D1/D2 leave behind. Because D3 uses the global `solver="scip"` path, these reference intervals are certified, not merely local optima—unlike `ExactInference`, which is only a local solver and is not a reliable oracle.

**Extension tightness (gap B) via `verify_lmc_factorization.py`.** The factorization verifier reconstructs each product joint from a choice of local extreme points and evaluates the LMC residual

$$P(x, y, s) P(s) - P(x, s) P(y, s) \approx 0$$

for every assertion (its `lmc_constraint_groups_vec` residuals). A residual that stays  $\leq \text{tol}$  after a tightening change confirms the scheme has not broken the independence structure while shrinking the set; a non-trivial residual that the LCN does *not* permit localizes a strong-extension (gap B) violation that D4/D5 must address. This check is solver-free and deterministic, so it is a stable regression gate.

**End-to-end.** On small instances, posterior marginals from CVE under each scheme are compared to the D3-style full-joint bounds for the query atom (the same certified global solve, with the query as objective). The recommended sequence is D1, then D2 with D3 as the per-instance certifier of gap A; D4/D5 are pursued only once the D3 comparison shows the residual gap is genuinely the strong-extension gap B rather than remaining local looseness.